



Linux Command Line Basics

Streams & Redirection | Pipes & Command Execution | Substitution, Jobs & Aliases

Starting and Ending a Linux Session

- Log in with username and password (console, SSH or terminal app)
- Prompt shows you are ready: \$ for normal user, # for root
- Check the session with whoami and tty
- End it with exit, logout or Ctrl+D

```
ssh student@server  
whoami  
tty  
exit
```

Standard Streams: stdin, stdout, stderr

- stdin (0): input, from the keyboard by default
- stdout (1): normal output, to the screen
- stderr (2): error messages, also to the screen
- Each stream has a file descriptor number

```
ls /etc           # stdout
ls /nofolder      # stderr
cat               # stdin: type,
                  # then Ctrl+D
```

Output Redirection (>, >>)

- > sends output to a file and overwrites it
- >> appends output to the end of the file
- The file is created if it does not exist
- The screen shows nothing: output went to the file

```
echo "Hello" > out.txt
echo "World" >> out.txt
cat out.txt
# Hello
# World
```

Input and Error Redirection (<, 2>)

- < takes input from a file instead of the keyboard
- 2> sends only error messages to a file
- 2>&1 sends errors to the same place as output
- Use 2>> to append errors

```
wc -l < fruits.txt  
ls /nofolder 2> err.txt  
cat err.txt  
ls /etc /nofolder > all.txt 2>&1
```

Pipes (|): Chaining Commands

- A pipe sends the stdout of one command to the stdin of the next
- Chain as many commands as needed
- Only stdout passes through the pipe, not stderr

```
ls /etc | wc -l  
cat fruits.txt | sort | uniq  
ps aux | grep ssh | head -n 3
```

tee: Output to File and Screen

- tee copies its input to the screen and to a file
- -a appends instead of overwriting
- Use it in the middle of a pipeline to save a step

```
ls | tee list.txt  
ls | tee -a list.txt  
sort f.txt | tee s.txt | head -n 3
```

Command Execution and Exit Status

- The shell finds the command in PATH and runs it
- `$?` holds the exit status of the last command
- 0 means success, non-zero means failure
- `&&` runs the next command only on success, `||` only on failure

```
ls /etc ; echo $?      # 0
ls /nofolder ; echo $? # 2
mkdir d && cd d
cd nodir || echo "failed"
```

Command-Line Editing Shortcuts

- Move and delete without the mouse
- Tab completes names, Up/Down browse earlier commands
- Ctrl+C cancels the running command

Ctrl+A	start of line
Ctrl+E	end of line
Ctrl+U	delete to start
Ctrl+K	delete to end
Ctrl+W	delete last word
Ctrl+L	clear screen
Tab	auto-complete

Quoting: Single, Double, Backslash

- Single quotes '...' keep everything literal
- Double quotes "..." allow \$variable and \$(command)
- Backslash \ escapes the next character only

```
NAME=Amit
echo '$NAME'      # $NAME
echo "$NAME"      # Amit
echo \$NAME       # $NAME
echo "Cost: \$5"
```

Command Substitution: \$(...) and Backticks

- \$(command) is replaced by the output of the command
- Backticks `command` do the same but are older style
- \$() is preferred because it can be nested

```
echo "Today is $(date +%A)"  
FILES=$(ls | wc -l)  
echo "Files: $FILES"  
echo "Year: `date +%Y`"
```

Job Control: jobs, bg, fg

- & at the end runs a command in the background
- Ctrl+Z suspends the running command
- jobs lists them; bg resumes in background; fg brings to foreground
- kill %n stops job number n

```
sleep 100 &  
sleep 200      # Ctrl+Z  
jobs  
bg %2  
fg %1  
kill %1
```

Aliases: Command Shortcuts

- `alias name='command'` creates a shortcut
- `alias` alone lists all aliases; `unalias` removes one
- Aliases last only for the session
- Add them to `~/.bashrc` to keep them

```
alias ll='ls -l'  
alias  
ll  
unalias ll
```

Command History

- history lists the commands you typed earlier
- !! repeats the last command; !n runs command number n
- Ctrl+R searches the history as you type
- Saved in ~/.bash_history; history -c clears it

```
history | tail -n 5
!!
!15
# Ctrl+R, then type
# part of a command
```